

Merkle Commitments for Arkworks

Alexander J. Havlin
alexander.havlin@epfl.ch
EPFL

Andrew Zitek-Estrada
andrew.zitek@epfl.ch
EPFL

June 2026

Abstract

Merkle commitment schemes are increasingly essential tools in the construction of post quantum secure cryptography in general and zero knowledge proof systems specifically. While widely accepted techniques are intuitive to describe, their apparent simplicity disguises the potential for critical parameterisation errors. Practitioners are pulled in two directions: oversized parameters make the scheme slow, while under configuration leads to concrete vulnerabilities in the higher level protocol. In this effort, we address these challenges leveraging an interface that bridges security definitions with scheme configuration. Both digest width and per-leaf salt cardinality are tweakable parameters for a concrete instantiation. We demonstrate their relationship in Arkworks to the size of the field by showing that undersizing these parameters leads to a degradation of the security claimed by a particular scheme. Additionally, we provide methodology for describing the relationship between security and performance for properties like binding and hiding. Finally, we provide machine-checked analysis of the binding property as described in this framework using Lean 4.

1 Contributions

Merkle commitments are essential to modern proof systems, but developers often have to balance performant systems with evolving security requirements and configurability, leaving underconfigured schemes vulnerable. We aim to lift that burden into infrastructure.

- **Main contribution:** we provide a rule for evaluating a Merkle commitment scheme’s binding and hiding security against a target security parameter λ , field $\mathbb{F}$, hash function (or random oracle) $\mathcal{H}$, and hasher implementation (Section 2).
- **Cost measurement:** we benchmark prover time, verifier time, and proof size across parameter combinations spanning three fields and two hash families, comparing desired security thresholds with their cost.
- **Machine checking our results:** we give the binding derivation formalised in Lean 4 [z-l26].
- **Batch opening consolidation:** we consolidate Arkworks’ batch opening onto CoSet pruning, carrying forward the first semester’s proof-size win over its prefix-pruning predecessor: roughly 360 kB down to roughly 33 kB at 4,096 opened leaves.

In two semesters of work we investigated the structure of Merkle trees and Merkle commitments in depth, and what does and does not work in our favour. Chronologically, we started by looking at feature requests against the Merkle tree implementation in Arkworks’ *crypto_primitives*, which already sketched out many missing features, evidenced by an RFC (Section A.1). We scoped the problem to optimal batched index-proof pruning. As a reference we used Chiesa and Yogev’s *Building Cryptographic Proofs from Hash Functions* [CY26], for its close agreement with our definitions and the rigour of its proofs. For batch proofs, the book outlines a pruning algorithm we call *CoSet pruning* (Section A.2). Arkworks’ *crypto_primitives* used a distinct algorithm for the same objective, which we call *prefix pruning* (Section A.3). That comparison found no statistically significant trade-off on prover or verifier time and significantly saved on proof size.

In the second semester we shifted to the proposed Arkworks design shift, aimed at unifying tools that had previously been specific to individual constraint systems into one IOR/polynomial IOP compiler (Section A.10). Our contribution there was scoped to evaluating Vector Commitments and their Merkle-commitment subset for the hiding and binding security they bring to the library. A VC scheme must satisfy λ -bit binding (Section 2) and, optionally, λ -bit hiding (Section 2). For a Merkle scheme, both properties are enforced through the underlying hasher, and an undersized hasher still compiles and passes every functional test, which is what makes the problem easy to miss. Our research surfaced concerns that the design shift had overlooked, and this report makes them explicit for developers and users constructing a hasher for a target hiding and binding security level.

2 Background

Traditionally, when we think of Merkle trees we think of arity 2 throughout, one hasher f at every level, and a simple binding claim tied to the root commitment. As protocols mature and the area is studied more deeply, the picture has broadened to varying arity, mixing hashers across levels, and different flavours of binding, each motivated by performance and recursion-friendliness. That shift complicates security analysis, and it is easy to lose track of details that a fixed arity-2, single-hasher tree used to settle by construction.

Definition 2.1 (Vector commitment scheme). A *vector commitment scheme* [CF13] over alphabet Σ and length ℓ is a triple $\text{VC} = (\text{Commit}, \text{Open}, \text{Check})$:

- $\text{Commit}(\mathbf{m}) \rightarrow (\text{cm}, \text{aux})$ binds a message $\mathbf{m} \in \Sigma^\ell$ to a short commitment cm and retains a state aux ;
- $\text{Open}(\text{aux}, I) \rightarrow \text{pf}$ authenticates an index set $I \subseteq [\ell]$;
- $\text{Check}(\text{cm}, I, \mathbf{a}, \text{pf}) \rightarrow \{0, 1\}$ decides whether pf authenticates the subvector $\mathbf{a}$ at I against cm , where for an honest committer $\mathbf{a}$ is exactly the committed message $\mathbf{m}$ restricted to the positions in I .

A scheme is *complete* (Definition 2.2) and *position-binding* (Definition 3.1). It may also implement optional extensions, namely hiding (Definition 4.1), updatability, aggregation, functional opening, and knowledge soundness, of which this report treats only hiding.

Definition 2.2 (Completeness). VC is *complete* if for every $\mathbf{m} \in \Sigma^\ell$ and every $I \subseteq [\ell]$ the honest Commit-Open-Check round trip accepts $\mathbf{m}_I$ with probability 1. For Merkle this is honest reconstruction of the root from an opening.

Definition 2.3 (Merkle commitment scheme). The *Merkle commitment scheme* [Mer88] $\text{MT} = (\text{MT.Commit}, \text{MT.Open}, \text{MT.Check})$ instantiates Definition 2.1 in the random-oracle model over a general Merkle shape G_ℓ of depth $d = \max_i d_i$, the deepest of the per-leaf depths d_i over leaves i : the three algorithms query a shared oracle f , the commitment is the tree root, and the committer state is the retained salts and labels. The binary tree is the arity-2 subset of this shape. Two parameters carry the security: the digest width κ governs binding and the salt size s governs hiding, with no tuning of one supplying what the other lacks.

Binding is required of every scheme in this report, and hiding matters only where the protocol built on top needs zero-knowledge. At the `MerkleHasher` trait of `ark-mt` the digest and salt widths are associated types, fixed by whoever writes the implementation, and nothing compares them against the size of the field. BabyBear at $\lambda = 128$ therefore gets a one-element digest over a ~ 30 -bit space and a one-element salt carrying ~ 30 bits of entropy. The book [CY26] reaches the same figure and observes that such an instantiation is not flawed as such, since a protocol may bound the prover’s Merkle queries far below the birthday threshold and argue security from its own soundness analysis. The trait gives no way to declare that bound, which is the role of h in the rule below.

The design rule. A protocol designer supplies five things: a field $\mathbb{F}$, a security level λ , a hash H , a choice of standard or hiding interface, and an adversary model, declared as the query-budget exponent h . Everything else is derived. From $(\mathbb{F}, \lambda, h)$ the parameter constructor produces

$$D = \left\lceil \frac{\lambda + 1 + 2h}{b} \right\rceil, \quad k = \left\lceil \frac{\lambda}{b} \right\rceil, \quad S = \left\lceil \frac{\lambda}{8} \right\rceil, \quad (1)$$

where $b = \lfloor \log_2 |\mathbb{F}| \rfloor$ is the conservative bits-per-element. Here D is the digest width in field elements, and k, S are the field- and byte-salt cardinalities. Each output is a checkable obligation. The constructor either meets it and emits the derived parameters, or it refuses. A designer can therefore neither quietly under-specify nor silently over-claim.

The justification is short. A $q \leq 2^h$ -query collision search succeeds against a κ -bit digest with probability about $q^2/2^{\kappa+1}$, so meeting $2^{-\lambda}$ needs $\kappa \geq \lambda + 1 + 2h$ bits, and D is that floor in field elements (Lemma A.1). The salt is attacked one guess at a time, so k and S give it λ bits of entropy in the hash’s native unit.

The query budget is where the adversary model enters, and it is the one input the constructor cannot derive: a prover that grinds a Fiat–Shamir transcript spends its whole query budget searching for a favourable challenge, which is why the designer declares h rather than the constructor inferring it.

The report follows the rule by property from here. Section 3 and Section 4 each carry their own definition, failure, and derived parameter, and Section 5 measures what the derived parameters cost to run.

3 Binding

Binding is a key property of any vector commitment scheme, and different schemes have different notions of it [CY26]: subvector position binding, its single-position specialisation, and the weak variant that requires an honestly generated commitment. We look at subvector position binding of a Merkle commitment scheme and show that it can be reduced to the collision resistance of the underlying hash function. For a given hash function, collision resistance gives us a bound that we can evaluate against the desired security parameter. Specifically, we show that this bound falls short of common target λ -bit collision resistance for single field elements in small fields. The size of the digest output by a concrete hasher is inextricably linked to the underlying field we operate over. This realisation led us to flag a bug in Arkworks in which the hasher implementation would output a single field element (digest size) regardless of the size of the underlying field.

Definition 3.1 (Subvector position binding). VC is *subvector position binding* if no efficient adversary outputs a commitment cm with two accepted openings $(I_b, \mathbf{a}_b, \text{pf}_b)$ that disagree at some $i \in I_0 \cap I_1$, except with probability $\text{negl}(\lambda)$. The commitment has no honest-generation requirement.

Proposition 3.2 (Binding from collision resistance). *Let $\text{MT}[H]$ be a Merkle commitment over a perfect binary tree of depth d whose leaf encoding is injective and whose leaf and internal-node calls are domain separated. If the shared κ -bit oracle receives at most q distinct queries during the complete experiment, then*

$$\text{Adv}_{\text{VC}, \mathcal{A}}^{\text{bind}}(\Theta) \leq \Pr[\text{oracle-trace collision}] \leq \frac{q(q-1)}{2^{\kappa+1}},$$

which meets $2^{-\lambda}$ when $q \leq 2^h$ and $\lambda + 1 + 2h \leq \kappa$. [Lean-backed]

The binary tree is the case Chiesa and Yorgev prove directly, and we take their proof as given [CY26]: either the adversary’s query trace already contains a collision, or the two accepted openings of the minimal disputed index yield one across the two verification traces, bounded by a union bound over the at most $(d+1)^2$ query pairs. This is the straightforward form the book states before improving on it, and the form Equation (1) consumes. The reduction is machine-checked in Lean 4 [z-l26] for a general Merkle shape, closed for singleton openings and argued on paper for a general index set (Section A.4).

Now that we have seen the reduction to collision resistance, consider how a hasher is implemented concretely. Every hasher in `ark-mt` satisfies one `MerkleHasher` trait (Section A.5), and the current Poseidon2

implementation sets its digest to one field element, for every field $\mathbb{F}$ and every target λ . Nothing in the trait or the implementation reads λ .

Now suppose we set a target security parameter $\lambda = 128$. BabyBear’s modulus is $p = 2013265921$, a 31-bit prime, and the conservative bound used throughout this report is $b = \lfloor \log_2 p \rfloor = 30$ bits per element, so a one-element digest publishes a root drawn from a 30-bit space. Applying Lemma A.1 at $\kappa = 30$, a collision search succeeds once the query count passes roughly $2^{15.5} \approx 46,000$, well short of a $\lambda = 128$ target.

The book states this result plainly for a 31-bit field. We ran the search only to confirm that the one-element preset sits in that regime, and it produced two trees over different messages sharing one root in 44 seconds on a commodity laptop, averaged over three seeded runs (Section A.8). The collision is cross-shape, between an 8-leaf binary tree and an 8-leaf 8-ary tree on the same Poseidon2 instance, and it still breaks Definition 3.1 as stated because `hash_leaf` and `hash_nodes` carry no layer, index, or arity tag (Lemma A.3). Working backwards from λ and $\mathbb{F}$, Equation (1) asks for $D = \lceil 257/30 \rceil = 9$ elements at $\lambda = 128$ and $h = 64$, so fixing the hasher means widening `Digest` from `F` to a nine-element array and hashing into all nine on every call, which we price in Section 5.

4 Hiding

A zero-knowledge protocol relies on more than hiding, and the book’s stronger requirement is *equivocation*, which a Merkle commitment satisfies in the random-oracle model through an inefficient simulator that programs the oracle, though never in the *trapdoor* form an efficient simulator would need, since there is no `Setup` secret to program (`td = \perp`) [CY26]. We work with the indistinguishability form of hiding below, which rests entirely on the salt space, and show that the current salt is too small to carry it.

Definition 4.1 (Hiding). A randomised VC whose `Commit` draws salts from a space $\mathcal{R}$ is *hiding* if, for any two messages of length ℓ and any adversary that after seeing `cm` adaptively opens positions on which the messages agree, the advantage in deciding which was committed is at most $\text{negl}(\lambda)$. For Merkle this refines to *root hiding*, meaning the real root is statistically close to one simulated without `m`, strengthened to *selective-opening hiding*, which must also hide through the disclosed salts and authentication paths of partial openings.

In a Merkle commitment scheme, we refer to salt simply as the random per-leaf nonce. Each leaf is itself a basic commitment $H(\mathbf{m}_i, \text{salt})$, and the salt travels in the authentication path beside the sibling digests so the verifier can recompute the leaf. Opening one leaf commitment under two different salts would force a hash collision, so the salt binds a value to its position even as it hides it. Hiding then rests entirely on the salt space, which must be both well sampled and large enough to resist search within the security budget, neither of which a deterministic salt satisfies, since the receiver can recompute candidate leaf commitments.

The digest and the salt scale differently in the query budget, which is what Equation (1) is tracking when it derives D from h and derives k and S from λ alone. A binding adversary wins on any collision among its own queries, so its advantage grows as $q^2/2^\kappa$ and the digest has to carry $2h$ bits of headroom. A hiding adversary has to hit one particular salted leaf input, so its advantage grows as $\ell q/2^s$, and the salt needs only λ bits.

Proposition 4.2 (Salt cardinality). *Let the salt be drawn from k elements of a field with $b = \lfloor \log_2 |\mathbb{F}| \rfloor$ bits per element, or from S bytes. Then $|\mathcal{R}| \geq 2^\lambda$ whenever $k \geq \lceil \lambda/b \rceil$ or $S \geq \lceil \lambda/8 \rceil$. [Lean-backed]*

This is what Equation (1) computes for k and S and the one part of hiding that is machine-checked, while the bound it feeds into remains an *intended reduction* (Section A.9).

Looking again at the Arkworks library, the current hiding hasher repeats the undersized-parameter mistake we saw for binding, this time in the salt space (Section A.5): the salt is one field element for every field and every target λ , drawn uniformly, so the sampling requirement is met and what falls short is the number of values it can take.

Take a target security parameter $\lambda = 128$ and BabyBear again. Each field element carries at most $b = \lfloor \log_2 p \rfloor = 30$ bits under the conservative accounting of Equation (1), so a one-element salt ranges over roughly 2^{30} values. An adversary who knows the two candidate messages recovers which was committed by hashing 2^{30} candidate leaves against the opened position. Reaching 2^λ instead needs $k = \lceil \lambda/b \rceil = \lceil 128/30 \rceil = 5$

Property	Field	Parameter	Prover (ms)	Verifier (ms)	Proof (kB)
Hiding	BabyBear	$k = 5$	25.7	1.03	3.6
Hiding	Goldilocks	$k = 3$	39.8	1.51	4.6
Hiding	BN254	$k = 1$	175.2	6.56	10.3
Binding	BabyBear	$D = 9$	38.7	1.84	10.1
Binding	Goldilocks	$D = 5$	40.3	1.37	11.0
Binding	BN254	$D = 2$	260.0	11.26	16.4

Table 1: Measured cost of the first Poseidon2 configuration reaching $\lambda \geq 128$ per field, for the hiding benchmarks (varying the salt cardinality k) and the binding benchmarks (varying the digest width D). Prover time is mean commit plus open; verifier time is mean check; proof size is the compressed proof. Fixed regime as in Figure 1: binary tree, 4,096 leaves, 32 openings.

elements. The same constructor gives $k = 3$ for Goldilocks at $b = 63$, and $k = 1$ for BN254 at $b = 253$, where a single element already carries more than the target.

5 Cost

What the benchmarks measure. The rule says which configurations support a security claim, and the benchmarks say what those configurations cost. Each plot fixes a binary tree with 4,096 leaves and 32 openings and reports mean prover time, verifier time, and proof size. A field-coloured line marks the first configuration reaching $\lambda \geq 128$, and a red $\times$ marks each cost transition.

The salt arrives in the hash’s native unit. The rule reports the salt in the hash’s own unit, k field elements for an algebraic hash such as Poseidon2 [GKS23], or S bytes for a byte-oriented hash such as Blake3 [OANWO20]. Because the unit already matches the hash, the salt pays no encoding penalty when it is absorbed.

Permutations set the transitions. Sponge-based hash functions operate on a *state* of size $t = \text{rate} + \text{capacity}$, where the capacity carries the previous round’s output and the rate is the room left to absorb new input. We measure how many elements a construction absorbs before it needs a second permutation, then overlay the security the rule certifies to mark where each field reaches the target.

When binding is free. Proof size grows linearly with the digest width for every field, so a user with proof size concerns can read off how much security a given size buys. Timing has a step shape instead: within a permutation level, widening the digest gains security at no measurable cost, and crossing a level raises prover and verifier time sharply before flattening again. A parent hash absorbs two child digests, so a digest is charged once per tree level while a salt is absorbed once at the leaf, which is why a binary tree stays inside one permutation only while $2D \leq t$. BabyBear’s Poseidon2 preset runs at $t = 16$ ($D \leq 8$ free) against a certified $D = 9$, and BN254 runs at $t = 3$ ($D \leq 1$ free) against a certified $D = 2$: both sit one element past the boundary and pay for a second permutation, taking prover time from 25.1 to 38.7 ms for BabyBear and 168.6 to 260.0 ms for BN254. Goldilocks is the exception: its $t = 12$ preset holds $D \leq 6$ while the rule asks for only $D = 5$, so its certified binding parameters sit inside a permutation block computed anyway (Figure 4, left). Proof size orders the fields the other way: Goldilocks reaches $\lambda = 128$ at fewer digest-width increments than BabyBear, yet its proof at the crossing is the larger of the two, 11.0 against 10.1 kB, because each Goldilocks element is wider.

Hiding is cheap everywhere. The rule asks for $k = 5$ field salts on BabyBear, $k = 3$ on Goldilocks, and $k = 1$ on BN254, while the measured leaf transitions fall at $k = 16$, $k = 12$, and $k = 3$ respectively. Every field therefore reaches $\lambda = 128$ well inside its first permutation block, and prover and verifier time stay flat as the salt widens. A user weighing hiding needs to consider only the proof size trade-off, which grows linearly in the number of salt elements.

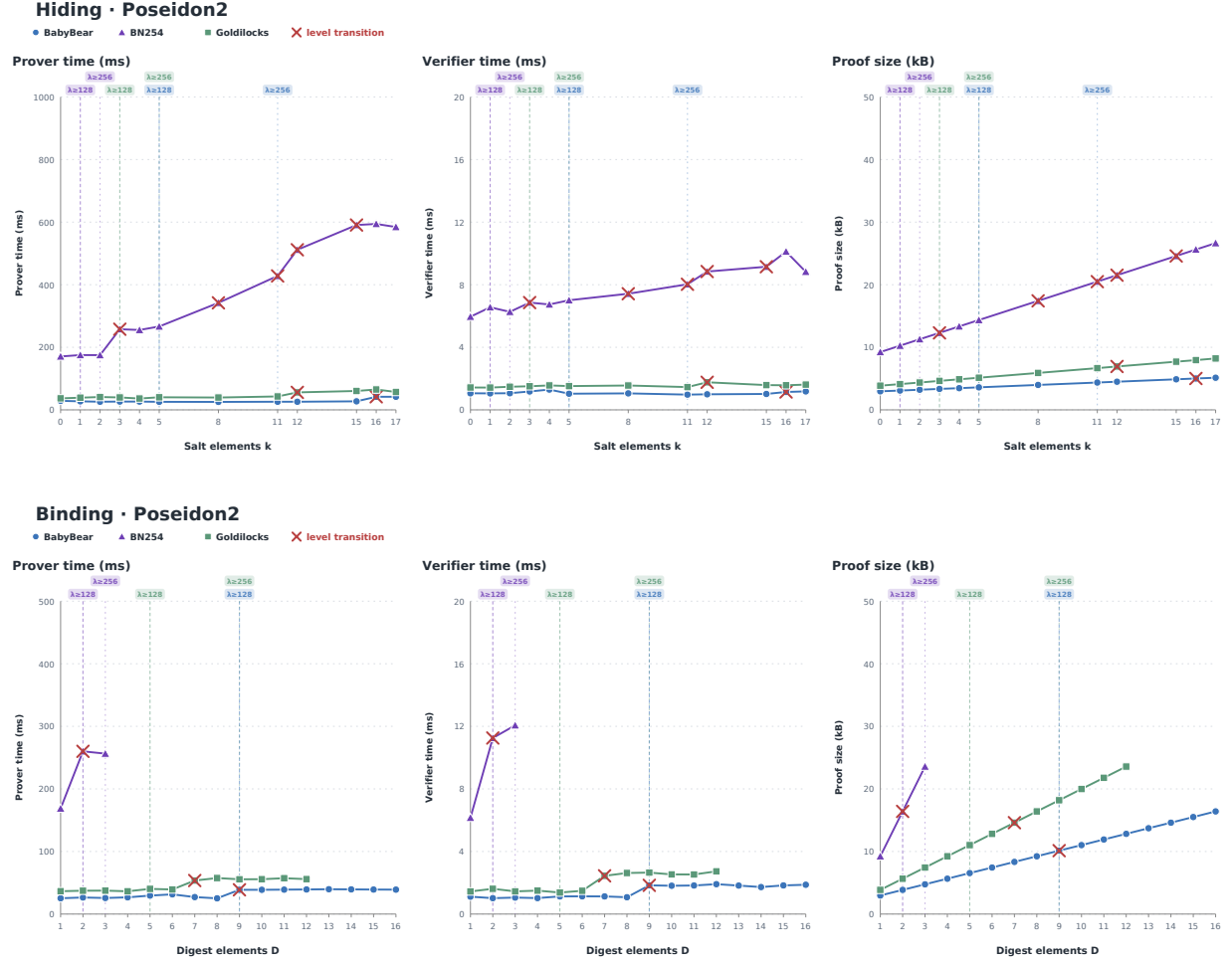


Figure 1: Poseidon2 hiding cost as salt width grows (top) and binding cost as digest width grows (bottom). Field-coloured lines mark the first measured $\lambda \geq 128$ point for BabyBear, Goldilocks, and BN254. Red x markers flag inputs that cross into an extra permutation.

Byte-wise hashers step at block boundaries. Blake3 compresses over fixed blocks, and an input that crosses a boundary needs a second compression call. Binding shows the same step structure as Poseidon2: the $\lambda = 128$ digest occupies 33 bytes, the first boundary, and crossing it takes prover time from 1.72 to 2.28 ms (Figure 4, right). Hiding clears its own target before the first transition, so its only cost is the proof size, which grows linearly with the salt width.¹ [empirical; binary tree, 4,096 leaves, 32 openings]

6 Conclusion and Next Steps

A vector commitment is only as strong as the parameters underneath it, and Arkworks’ current defaults miss that target for small fields on both axes at once. The rule of Equation (1) derives, from a designer’s declared inputs, the digest and salt parameters a security target forces, and refuses the undersized configurations those defaults allow, including the one broken here in 44 seconds. The cost benchmarks price the derived parameters and separate the security thresholds from the cost transitions. For Poseidon2 the certified hiding salts fit inside permutation blocks already paid for on all three fields, so hiding at $\lambda = 128$ costs nothing extra. Binding is free in the same sense only on Goldilocks. BabyBear and BN254 reach their certified digest widths one element past the permutation boundary and pay roughly half again in prover time for it. That

¹The full byte-wise Blake3 benchmark results are plotted in Figure 5 (Section A.7). The prose here summarises their result.

cost is what the correct parameter is worth on those fields. The binding reduction and the salt-cardinality lemmas hiding needs both carry a machine-checked Lean backing.

Next. The formal status splits three ways, and the split is the information. Closed: the binding chain end to end, and the salt-cardinality lemmas hiding needs. Scoped and believed but not yet closed: the extraction bridge and one distributional averaging step, both reduced to named deterministic pieces. Not yet stated in the formal model: the sampling-based hiding game and equivocation, so the hiding claim rests on the intended reduction of Section 4 rather than a checked theorem. Beyond the formalisation, multi-round BCS and Fiat–Shamir soundness would justify the strengthened digest target at the protocol level, and the k -ary and arbitrary-length shapes share the generic binding reduction but are not yet exhibited.

A Appendix

A.1 The originating RFC

This work began as implementation work around open feature requests against the `Arkworks/crypto-primitives` Merkle tree: simplify the split hash interface, remove caller-side ordering footguns, support richer tree shapes, and compress multi-openings. Read together, those requests pointed to one broader issue. The implementation had many locally reasonable decisions, but no single specification against which their combined behaviour could be reviewed.

A.2 CoSet pruning

The book [CY26] states the optimisation as `MT.CoSet`. Working layer by layer, for each opening we look at the nodes for a single proof and apply two optimisations: deduplication, and exclusion of computable nodes.

Figure 3 works the case of openings $\{2, 5\}$ on a 16-symbol tree whose leaves each hold a coset of two symbols, so index 2 is carried by leaf I and index 5 by leaf J . The two single-index co-paths are

$$\text{CoPath}(I) = [H, E, C], \quad \text{CoPath}(J) = [K, D, C].$$

Deduplication removes the second copy of C , which both paths need. Exclusion removes D and E , because each is computable by the verifier: D is the hash of H and the opened leaf I , and E is the hash of the opened leaf J and K . The proof is therefore $[H, K, C]$, three nodes against the six that two separate authentication paths would carry.

Knowing which vertices matter leaves the question of how to label them, and we implemented and benchmarked both answers. `CoSet` carries explicit vertex labels. `ImplicitCoPath` derives the same labels from the opened indices, performing the same pruning without materialising a coordinate bookkeeping layer. The `Committed/OpeningProof/Opening` split that came with them makes ordering and duplicate-index validation part of construction. Figure 2 gives the measurement, and the benchmark harnesses built to take it prefigure the cost benchmarks of Section 5.

A.3 Prefix pruning

Prefix encoding is the baseline pruning algorithm in `Arkworks/crypto-primitives`. It constructs the full authentication path for every opened index and then compresses each path against its immediate predecessor:

1. Initialise the set of leaf indices I .
2. For each index $i_t \in I$, add the sibling of i_t to the proof `pf` and let P_t be the full authentication path of i_t .
3. Set $\text{prefix_len}_t \leftarrow |\text{lcp}(P_{t-1}, P_t)|$ and $\text{suffix}_t \leftarrow P_t[\text{prefix_len}_t:]$.
4. Add prefix_len_t and suffix_t to `pf`.

Two properties follow. The encoding pays for full paths even where many of them overlap, and it assumes a previous path in a fixed order, which leaves the encoding ambiguous unless that order is pinned by the caller. `CoSet` pruning reaches the same set of necessary nodes without either property, which is what Section 1 reports as the proof-size win.

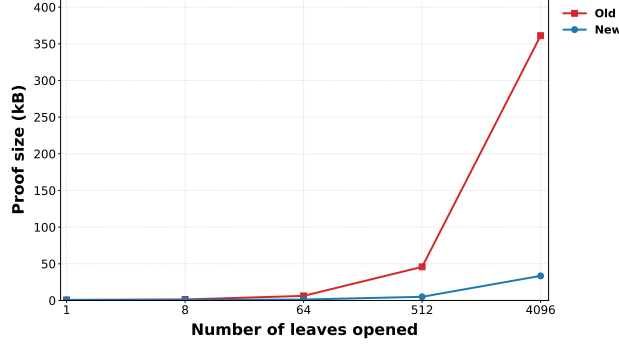


Figure 2: Phase-1 multiproof compression on clustered openings of a 4,096-leaf tree. The CoSet encoding prunes shared path material, so proof size tracks the size of the disclosed frontier. At 4,096 opened leaves the compressed proof is roughly 33 kB against roughly 360 kB uncompressed.

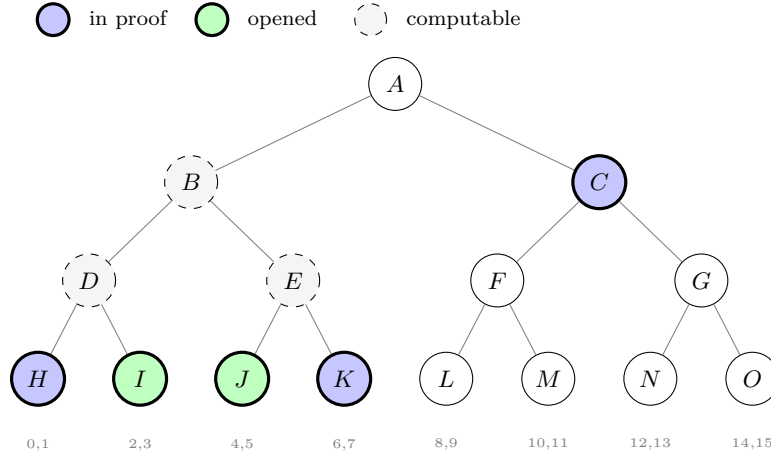


Figure 3: CoSet pruning for openings $\{2, 5\}$. Each leaf holds a coset of two symbols, so the openings are carried by leaves I and J . Deduplication keeps one copy of C , and D , E , and B are excluded as computable from the opened leaves and the nodes already sent. The proof is $[H, K, C]$.

A.4 Binding for a general Merkle shape

Proposition 3.2 is stated in Section 3 for a perfect binary tree, which is the case Chiesa and Yogev prove directly [CY26]. The same bound holds over a general Merkle shape G_ℓ , and this section gives that argument.

The argument runs through three lemmas.

Lemma A.1 (Birthday bound). *Let f be drawn uniformly from the functions into $\{0, 1\}^\kappa$, and suppose q distinct inputs are queried. The probability that two of the answers coincide is at most $q(q-1)/2^{\kappa+1}$.*

Proof. The q answers are independent and uniform on $\{0, 1\}^\kappa$, because the inputs are distinct and f is uniform. Fix an unordered pair of query indices. Its two answers agree with probability $2^{-\kappa}$. There are $\binom{q}{2} = q(q-1)/2$ such pairs, so a union bound over them gives $q(q-1)/2 \cdot 2^{-\kappa} = q(q-1)/2^{\kappa+1}$. $\square$

Lemma A.2 (A break extracts a collision). *Suppose MT.Check accepts both $(I_0, \mathbf{a}_0, \text{pf}_0)$ and $(I_1, \mathbf{a}_1, \text{pf}_1)$ against the same commitment cm , and the two disagree at some $i \in I_0 \cap I_1$. Then the oracle trace of the experiment contains two distinct inputs with the same output.*

Proof. Acceptance of opening b means the verifier recomputes a labelled path from leaf i to the root. Write that path $\ell_0^b, \ell_1^b, \dots, \ell_d^b$, where ℓ_0^b is the leaf label and $\ell_d^b = \text{cm}$. Each ℓ_0^b is the output of a leaf-tagged call on the encoded symbol at i , and each ℓ_{j+1}^b is the output of a node-tagged call whose input contains ℓ_j^b .

The two openings disagree at i , and the leaf encoding is injective, so the two leaf-tagged calls have distinct inputs. If $\ell_0^0 = \ell_0^1$ those two distinct inputs already share an output, which is the claimed collision. Otherwise

$\ell_0^0 \neq \ell_0^1$ while $\ell_d^0 = \ell_d^1$, so the two label sequences disagree at level 0 and agree at level d . Let j be least with $\ell_{j+1}^0 = \ell_{j+1}^1$, which exists because level d agrees. By minimality $\ell_j^0 \neq \ell_j^1$. Both paths pass through index i , so on both sides ℓ_j^b occupies the same child slot of the level- $(j+1)$ call. The two level- $(j+1)$ inputs therefore differ while their outputs agree. Domain separation makes the colliding pair genuine, because the leaf and node call families carry distinct tags and this pair is drawn from within one family. $\square$

Lemma A.3 (Shape is pinned only by geometry tags). *If node calls absorb no geometry tag, then two shapes whose roots collide admit openings that each shape’s own verifier accepts against the other’s root. If every leaf and node call instead absorbs its position as a tag of layer, index, and arity, then a root accepted under shape G is accepted under a shape $G' \neq G$ only if the trace contains a collision.*

Proof. For the first part, an untagged root is the output of the topmost call and its input records nothing about the shape that produced it, so any pair of shapes reaching a common root admits both readings. The demonstration below exhibits such a pair for arity 2 against arity 8. For the second part, acceptance under G forces, at every position on the opening path, a call whose input begins with that position’s tag under G . A shape $G' \neq G$ differs from G at some position on the path in layer, index, or arity, so the corresponding two calls have distinct inputs. Both outputs are the accepted root, which is a collision. $\square$

Proof of Proposition 3.2. Let $\mathcal{A}$ win the position-binding experiment, so it outputs cm with two accepted openings that disagree at a shared index. Lemma A.2 converts that pair into two distinct inputs of the trace sharing an output. Lemma A.3 removes the remaining freedom, since under the stated domain separation any re-reading of the same digests as a different shape also leaves a collision in the same trace. Every winning execution therefore leaves a collision behind, which gives $\text{Adv}_{\text{VC}, \mathcal{A}}^{\text{bind}}(\Theta) \leq \Pr[\text{oracle-trace collision}]$. The complete experiment issues at most q distinct queries, so Lemma A.1 bounds that probability by $q(q-1)/2^{\kappa+1}$. For the concrete condition, $q \leq 2^h$ gives $q(q-1) < 2^{2h}$, hence

$$\text{Adv}_{\text{VC}, \mathcal{A}}^{\text{bind}}(\Theta) < \frac{2^{2h}}{2^{\kappa+1}} = 2^{2h-\kappa-1},$$

and $2^{2h-\kappa-1} \leq 2^{-\lambda}$ holds if and only if $\kappa \geq \lambda + 1 + 2h$. $\square$

The reduction is machine-checked in Lean 4 [z-l26] for each statement. Lemma A.1 is `birthdayBound`, proved by union bound over the $q(q-1)/2$ unordered pairs, and specialised to κ -bit digests as `birthdayBound_kappa`. Lemma A.2 is `mt_colliding_paths`, which rests on `walkCopath_inj` for the injectivity needed to walk upwards in the tree, and its two hypotheses are supplied by `encodeLeaf_inj` and `encodeLeaf_ne_encodeNodes`. The step from a winning adversary to a trace collision is `coupling_trace_le_collisionBound`, and the closing arithmetic is `collisionBound_le_inv_pow`. The rule is carried as a decidable definition, `MeetsBindingTarget` over $\kappa = \text{fieldBits} \cdot \text{digestElems}$, with `meetsBindingTarget_offield` discharging it for any field and `babyBear_meetsBindingTarget` discharging BabyBear with no hypothesis at all.

Two caveats on that scope. The formalisation closes the singleton-opening case end to end, and the multi-leaf collision lemma is still in progress, so Lemma A.2 is mechanised for a single opened index and argued on paper for a general index set. Lemma A.3 is not mechanised. The reduction is otherwise generic over the Merkle shape, with the binary tree as the exhibited instance, and k -ary and arbitrary-length shapes share the argument without yet being exhibited (Section 6).

A.5 The MerkleHasher trait

Every hasher in ark-`mt` satisfies one trait, with the digest and salt widths left as associated types:

```
pub trait MerkleHasher: Clone + Send + Sync {
  type Symbol: Clone + Send + Sync;
  type Digest: Clone + Eq + Send + Sync + WireEncode + WireDecode;
  type Salt: Clone + Default + Send + Sync + WireEncode + WireDecode;

  fn sample_salt<R: RngCore + CryptoRng>(&self, rng: &mut R) -> Self::Salt;
  fn hash_leaf(&self, symbol: &Self::Symbol, salt: &Self::Salt) -> Self::Digest;
  fn hash_nodes(&self, children: &[Self::Digest]) -> Self::Digest;
}
```

The Poseidon2 hasher sets the digest to be the field it is instantiated over, and its hiding counterpart samples the salt the same way:

```
impl<F: PrimeField> MerkleHasher for Poseidon2Hasher<F> {
    type Digest = F;
    ...
}

impl<F: PrimeField + Absorb> MerkleHasher for PoseidonHidingHasher<F> {
    type Salt = F;

    fn sample_salt<R: RngCore + CryptoRng>(&self, rng: &mut R) -> F {
        F::rand(&mut ark_rng_wrapper(rng))
    }
    ...
}
```

Neither implementation reads the target security parameter λ . The digest and salt widths are whatever the associated types resolve to, fixed once at the type level and never checked against the field.

A.6 Why the cost transitions fall where they do

Section 5 prices what crossing a permutation or compression-block boundary costs. This figure is the mechanism behind that step.

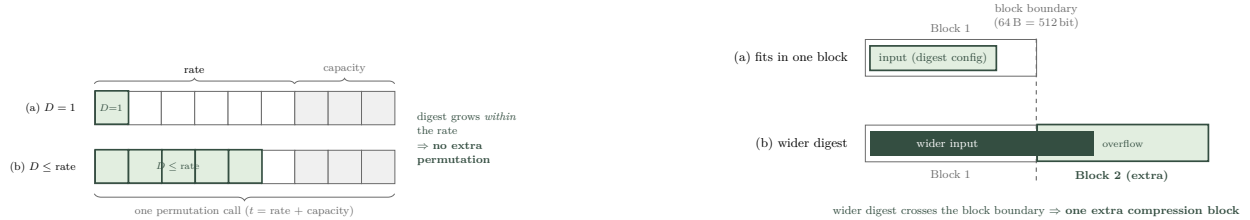


Figure 4: Why the cost transitions of Section 5 fall where they do. Left: a Poseidon2 permutation writes its output into the rate of a fixed-width state, so widening the digest up to the rate stays inside one permutation call and is free. Right: Blake3 absorbs in fixed blocks, so a digest that crosses a block boundary costs one extra block.

A.7 Byte-wise cost curves

The Blake3 counterpart to Figure 1, in the same triptych format:

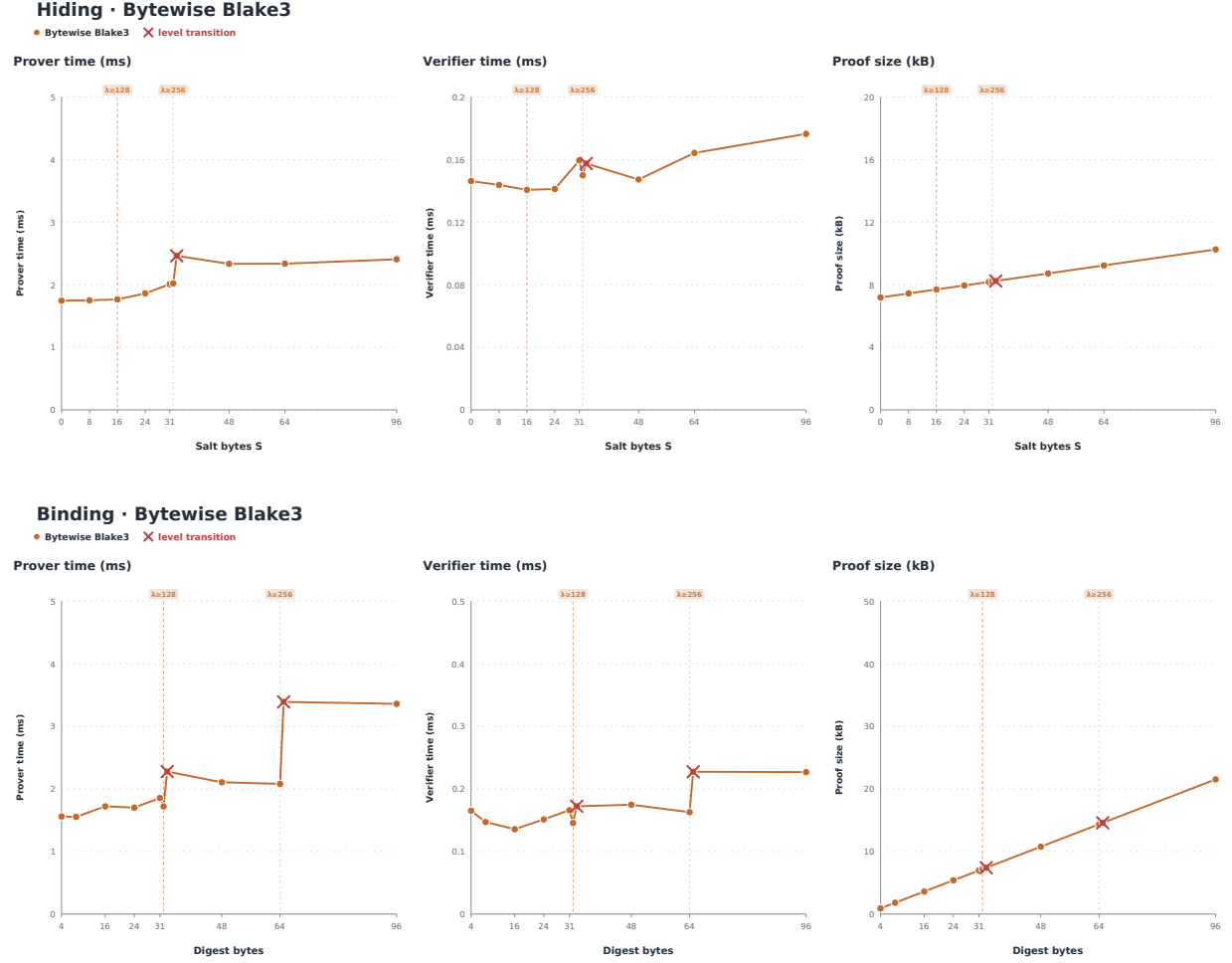


Figure 5: Blake3 hiding cost as salt width grows (top) and binding cost as digest width grows (bottom), both measured in bytes. Markers flag inputs that cross into an extra compression block.

A.8 The collision demonstration

The search of Section 3 precomputes the roots of one shape into a table and then draws from the other until a root repeats:

```
let mut binary_roots = HashMap::with_capacity(PRECOMPUTE_BUDGET); // 2^16
for _ in 0..PRECOMPUTE_BUDGET {
    let message = random_message(&mut rng);
    let committed = binary.commit(&message);
    binary_roots.insert(root_key(committed.root()), BinaryCandidate { message });
}
for searched in 1usize.. {
    let kary8_message = random_message(&mut rng);
    let kary8_committed = kary8.commit(&kary8_message);
    if let Some(binary_candidate) = binary_roots.get(&root_key(kary8_committed.root())) {
        // binary_candidate.message and kary8_message now share a root
        ...
    }
}
```

A.9 Hiding: the intended reduction

Proposition 4.2 certifies the salt cardinalities that Section 4 uses. This section states the bound they feed into and what remains open.

The cardinalities themselves are machine-checked, as `fieldSalt_card_lower_bound` and `byteSalt_card_lower_bound`, with `babyBear_byte_salt_hiding` as the concrete instance [z-l26]. The bound they feed into remains an *intended reduction*. The root-hiding obligation has the form

$$\text{Adv}_{\text{VC}, \mathcal{A}}^{\text{hide}}(\Theta) \leq \frac{\ell q}{2^s} + \varepsilon_{\text{internal}}(\kappa, q, \text{shape}),$$

where the two terms represent the two bad events, a query to a hidden salted leaf input, or an internal hash collision as the simulation rebuilds the tree from the leaves up to the root. The bound is written against the intended game, in which the honest experiment samples the oracle jointly with the salts, and discharging that game is the open step. The salt-hit term rests on a standard averaging step that is not yet formalised. The certified salt cardinalities are prerequisites for this bound and not the theorem itself, and lifting the result to the selective-opening target is the composition step after that.

A.10 Compiled soundness

In the compiler that `ark-vc` [Ark26] targets, each oracle message is committed and the verifier’s queries are answered with authenticated openings. The root is fixed into the transcript from the first message, so *binding* enters the compiled soundness bound additively [CDGS24], which is what lets a soundness argument treat each committed answer as fixed before the query set is chosen. *Hiding* is what a zero-knowledge argument relies on across the same compilation, and its error grows only linearly in the query budget, which is why Section 4 sizes the salt from λ alone. The grinding adversary that inflates the query budget, and the digest target it forces, are priced in Section 3.

References

- [Ark26] Arkworks contributors. `ark-vc`: Vector commitment interfaces and Merkle instantiations. <https://github.com/arkworks-rs/z/>, 2026. Crate under development in the `arkworks` project.
- [CDGS24] Alessandro Chiesa, Marcel Dall’Agnol, Ziyi Guan, and Nicholas Spooner. On the security of succinct interactive arguments from vector commitments, September 2024. Manuscript.
- [CF13] Dario Catalano and Dario Fiore. Vector commitments and their applications. In *Public-Key Cryptography (PKC)*, volume 7778 of *LNCS*, pages 55–72, 2013.
- [CY26] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2026. Draft.
- [GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In *AFRICACRYPT 2023*, volume 14064 of *LNCS*, pages 177–203, 2023.
- [Mer88] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO ’87*, volume 293 of *LNCS*, pages 369–378. 1988.
- [OANWO20] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. BLAKE3: One function, fast everywhere, 2020.
- [z-l26] z-lean contributors. `VectorCommitment`: Executable Merkle commitments and security proofs in Lean. <https://github.com/z-tech/z-lean/>, 2026.